

編輯



對話交接文件

1. 主題與目標

本對話原始目標是將使用者現有的 Google Sheets 投資資料整理成網頁系統

經過討論後，專案目前已收斂為：

> 開發一個「台股族群成交值排行榜變化」網頁，透過每日歷史資料，判斷近

預計支援不同觀察週期：

- 1 個交易日
- 近 5 個交易日
- 近 10 個交易日
- 近 20 個交易日
- 近 60 個交易日
- 未來可支援自訂交易日區間

「月」原則上建議定義為 20 個交易日，而非日曆月，以避免每個月交易日數

目前暫緩的範圍：

- 概念股知識圖譜
- 完整產業供應鏈地圖
- 族群新聞時間軸
- 多來源新聞連結管理
- AI 新聞摘要
- 交易紀錄與資產配置 Dashboard
- 盤中即時行情
- 手機 App
- 買賣訊號

2. 使用者背景

工作與技術背景

- 使用者是軟體工程師。
- 約有 8 年工作經驗。
- 工作主要使用：
 - Windows
 - Visual Studio
 - C#
 - WPF
- 少部分工作經驗使用過 C++。
- 主要工作領域：
 - 機器視覺
 - 自動化應用
 - SPI／PCB 檢測設備相關系統

個人開發環境

此專案會在個人電腦上開發：

- 作業系統：macOS
- 編輯器：VS Code
- 使用者對 macOS+VS Code 的網頁開發流程不熟悉
- 教學時需以新手／小白角度拆解
- 不應假設使用者熟悉：
 - CLI 指令
 - ASP.NET Core
 - Blazor
 - Entity Framework Core
 - 資料庫 Migration
 - Google Cloud／Google API 認證
 - Git 指令流程
 - 網頁前後端架構

雖然使用者有 C# 和 WPF 經驗，但網頁開發部分仍需逐步說明：

1. 指令在哪裡輸入
2. 每個指令的用途
3. 預期看到什麼結果
4. 檔案應建立在哪個資料夾
5. 程式碼應貼到哪個檔案
6. 執行失敗時如何檢查

3. 現況摘要

原始 Google Sheets

使用者提供的 Google Sheets：

<https://docs.google.com/spreadsheets/d/1rZpWz5GSHkvjoXlp5mX6>

先前對話中辨識到的分頁包括：

- 每日新聞
- 故事
- 上市櫃名單 & 族群
- 概念股
- 筆記
- 出貨旺季
- 操作（台）
- 操作（台）_韭菜
- 損益－操作（台）
- 損益－存股（台）
- 損益－存股（美）
- 資產配置
- 投信持有率
- 待辦事項

Google Sheets 原有瓶頸

使用者明確提出：

1. 族群分類／概念股無法有系統地連結與整理。
2. 族群性新聞無法依時間持續加入多個連結。
3. 族群分類／概念股的外部連結查詢性差。
4. 新增或改變族群分類／概念股很麻煩。

先前分析認為，這些問題主要來自：

- Google Sheets 是二維表格。
- 公司、族群、概念和新聞實際上是多對多關聯。
- 同一家公司可能屬於多個族群或概念。
- 同一新聞事件可能關聯多個族群、公司和來源。
- 將多個股票或多個網址放在同一儲存格，難以搜尋、統計和維護。

專案方向變更

原本討論過建立「個人投資研究知識庫」，包含：

- 公司
- 族群
- 概念
- 公司與分類關聯
- 分類與分類關聯
- 新聞事件
- 多新聞來源
- 外部研究連結
- 分類歷史

但使用者後來明確表示：

> 先暫緩概念股。

原因是已有其他網站提供相關產業結構及概念股內容，可先作為參考。

參考網站：

<https://aistockmap.com/?topic=advanced-packaging-test&active>

目前優先功能改為：

> 族群成交值排行榜與歷史排行變化。

4. 已確認事實

使用情境

- 第一階段是個人使用。
- 在 Mac 本機開發。
- 目前不需要多人協作或帳號系統。
- 現階段不是要做公開商業網站。
- 主要分析台股上市及上櫃股票。
- 核心分析頻率是日線／收盤後。
- 第一版不需要盤中即時更新。

資料分工方向

過渡期間仍保留 Google Sheets。

目前較適合的分工為：

Google Sheets

暫時管理相對靜態、人工維護的資料：

- 族群名稱
- 族群包含哪些股票
- 股票代號
- 股票名稱
- 是否啟用
- 族群備註

Google Sheets 可視為：

> 族群定義與成分股管理介面。

先前建議的基本格式：

GroupKey	族群名稱		股票代號		股票名稱		啟用	
---	---		---		---		---	
advanced-packaging	先進封裝		3131		弘塑		TRUE	
advanced-packaging	先進封裝		3583		辛耘		TRUE	
facility-engineering	廠務工程		2404		漢唐		TRUE	

後端資料庫

所有會隨時間持續累積的資料應存入後端資料庫：

- 每日個股行情
- 每日個股成交值
- 每日族群統計
- 每日族群排名
- 每日成分股貢獻
- 資料匯入紀錄
- 每日計算時使用的族群成員狀態

簡化理解：

> Google Sheets 定義「族群現在包含哪些股票」；資料庫記錄「每天市場發

資料來源方向

上市股票資料：

- 優先使用臺灣證券交易所 TWSE 官方資料。

上櫃股票資料：

- 優先使用證券櫃檯買賣中心 TPEx 官方資料。

第一版建議收盤後每日更新一次，不處理即時串流。

5. 已完成事項

目前尚未真正開始撰寫專案程式。

已完成的是需求及架構討論，包括：

- 確認 Google Sheets 的主要瓶頸。
- 討論過完整投資研究知識庫的資料模型。
- 決定暫緩概念股及知識圖譜。
- 將第一版目標改為族群成交值排行榜變化。
- 初步決定 macOS 上的技術工具方向。
- 初步定義 Google Sheets 與 SQLite 的責任。
- 初步定義排行榜需要的核心指標。
- 初步提出資料表結構。
- 初步提出開發階段順序。

尚未完成：

- 尚未建立 Git Repository。
- 尚未建立 Blazor 專案。
- 尚未建立 SQLite 資料庫。
- 尚未安裝 EF Core 套件。
- 尚未建立 Google Sheets API。
- 尚未串接 TWSE／TPEx。
- 尚未匯入歷史行情。
- 尚未完成任何網頁畫面。
- 尚未實際驗證公式。
- 尚未確定 Google Sheet 最終欄位格式。

6. 重要決策與理由

決策一：暫緩概念股與知識圖譜

原因：

- 市場上已有類似網站可供查詢。
- 概念股的關聯管理範圍大，第一版開發成本較高。
- 使用者目前更需要判斷哪些族群正在出現成交熱度。
- 成交值排行的輸入、計算和輸出都較明確。
- 每日資料累積後，系統價值會自然提高。

決策二：第一版做日線／收盤後資料

原因：

- 使用者要比較 5、10、20、60 日的趨勢。
- 盤中即時資料對第一版價值有限。
- 即時行情會增加：
 - 資料授權問題
 - 更新頻率
 - API 穩定性
 - WebSocket
 - 快取
 - 大量寫入
 - 錯誤補償
- 收盤後每日更新已足以完成核心分析。

決策三：Google Sheets 繼續作為過渡期資料來源

原因：

- 使用者已熟悉 Google Sheets。
- 現有族群分類資料已存在。
- 第一版不需要先製作完整的族群管理後台。
- 可降低初期搬移與資料清理成本。

但應避免讓同一筆資料在 Google Sheets 和資料庫同時被雙向修改。

決策四：新資料及歷史數據存入後端資料庫

原因：

- 排行變化需要持續累積每日歷史。
- Google Sheets 不適合大量時間序列資料及複雜查詢。
- 必須保存當日計算結果，避免歷史排名隨日後族群名單變更而改變。

決策五：先用 SQLite，不先用 PostgreSQL

原因：

- 單人、本機使用。
- SQLite 不需啟動資料庫伺服器。
- 資料儲存在單一檔案。
- 適合 macOS 與新手。
- 可使用 Entity Framework Core。
- 未來若部署到網路，再考慮 PostgreSQL。

決策六：使用 C# 為主的網頁技術

目前建議：

- .NET 10
- Blazor Web App
- Interactive Server
- ASP.NET Core
- Entity Framework Core
- SQLite

原因：

- 使用者主要技術是 C#。
- 不需同時學習 React、TypeScript、Node.js。
- 前端互動與後端資料處理可使用同一語言。
- Google 認證資訊可留在 Server 端，不會暴露到瀏覽器。

決策七：先做表格，不先做漂亮圖表

第一個可運作版本優先完成：

- 族群排行榜表格
- 期間切換
- 排名變化

- 族群明細
- 成分股貢獻

待資料與公式驗證正確後，再加入：

- 折線圖
- 熱力圖
- 趨勢圖
- 儀表板

7. 技術細節

7.1 建議工具

必要工具

- macOS
- VS Code
- C# Dev Kit
- .NET 10 SDK
- Blazor Web App
- ASP.NET Core
- Entity Framework Core
- SQLite
- Google Sheets API
- Git
- GitHub 私有 Repository

建議但非第一天必要

- DB Browser for SQLite 或 SQLiteStudio
- GitHub Desktop
- xUnit
- Bruno 或 Postman

第一版暫時不要加入

- React
- Next.js
- Vue
- TypeScript

- Docker
- PostgreSQL
- Redis
- Kubernetes
- 微服務
- 獨立前後端專案
- 複雜 Clean Architecture

7.2 Blazor 模式

建議建立：

```
```text
```

Blazor Web App

Interactivity: Server

Interactivity location: Global

Authentication: None

使用 Interactive Server，而非 WebAssembly。

預期架構：

瀏覽器

↓

Blazor Server/ASP.NET Core

├ Google Sheets API

├ TWSE/TPEX 資料來源

└ SQLite

## 7.3 專案結構原則

第一版使用單一 Solution、單一 Web Project。

不要一開始拆成多個專案或微服務。

建議資料夾：

Solution

└ Web Project

├ Components

| └ Layout

| └ Pages

| └ Shared

├ Domain

| └ Stocks

```
| └─ Groups
└─ Data
 | └─ AppDbContext.cs
 | └─ Configurations
 | └─ Migrations
└─ Infrastructure
 | └─ GoogleSheets
 | | └─ GoogleSheetsClient.cs
 | └─ Database
└─ Services
└─ wwwroot
└─ Program.cs
└─ appsettings.json
```

目前確定可保留的類別名稱：

- `AppDbContext`
- `GoogleSheetsClient`

其餘 Service 和 Repository 名稱尚未定案。

## 7.4 核心排行榜概念

不能只做「總成交值最大」排行。

大型電子族群可能長期占據前段，但不代表資金近期正在加速流入。

因此至少區分：

### 成交熱度排行

回答：

最近哪個族群吸收最多成交值？

公式：

近  $N$  日平均每日成交值  
= 近  $N$  個交易日族群總成交值  $\div$  實際交易日數

### 資金加速排行

回答：

哪個族群近期的成交值相較前期快速放大？

例如選擇 5 日：

- 本期：最近 5 個交易日
- 前期：再往前 5 個交易日

公式：

成交值倍數

$$= \text{本期平均每日成交值} \div \text{前期平均每日成交值}$$

或顯示增減率：

成交值增減率

$$= (\text{本期平均每日成交值} - \text{前期平均每日成交值}) \div \text{前期平均每日成交值}$$

## 7.5 排名變化

公式：

排名變化

$$= \text{前期排名} - \text{本期排名}$$

例如：

前 5 日期間排名：第 8 名

最近 5 日排名：第 1 名

$$\text{排名變化} = 8 - 1 = +7$$

意義：

- 正數：排名上升
- 負數：排名下降
- 0：排名不變

## 7.6 市場成交比

公式：

族群市場成交比

$$= \text{族群成交值} \div \text{全市場股票成交值}$$

原因：

若族群成交值增加 30%，但整體市場成交值增加 50%，該族群實際取得的市場注意力反而下降。

因此應同時觀察：

- 絕對成交值
- 市場成交比
- 市場成交比的期間變化

注意：

同一家公司可能存在於多個族群，因此族群市場成交比加總可能超過100%。

這不一定是錯誤，而是族群分類重疊的結果。

## 7.7 上漲家數比

公式：

上漲家數比

$$= \frac{\text{期間內上漲的有效成分股數}}{\text{有有效行情資料的成分股數}}$$

用途：

- 判斷是整個族群同步上漲。
- 或只是少數大型股票帶動。

## 7.8 前三大成交集中度

公式：

前三大成交集中度

$$= \frac{\text{族群內成交值最高的前三檔股票成交值合計}}{\text{族群總成交值}}$$

用途：

- 高集中度：可能只有少數個股在動。
- 低集中度：資金較廣泛擴散到族群成分股。

## 7.9 建議核心指標

第一版建議呈現：

1. 平均每日成交值
2. 成交值增減率／成交值倍數
3. 排名變化
4. 市場成交比

5. 族群期間報酬率
6. 上漲家數比
7. 前三大成交集中度
8. 成分股成交值貢獻

其中「族群期間報酬率」的算法尚未確認。

待決定：

- 等權平均
- 成交值加權
- 市值加權
- 使用族群指數
- 其他方式

## 7.10 排行頁初步欄位

排名

排名變化

族群

近  $N$  日平均成交值

較前  $N$  日成交值增減率

市場成交比

市場成交比變化

族群期間漲跌

上漲家數比

前三大成交集中度

## 7.11 族群明細頁初步內容

點入族群後顯示：

目前排名

前期排名

排名變化

近  $N$  日平均成交值

較前  $N$  日成交值增減

市場成交比

族群期間報酬

上漲家數比

前三大成交集中度

成分股數

成分股貢獻表：

股票代號  
股票名稱  
近 N 日成交值  
族群成交值占比  
期間漲跌  
成交值增減

未來可加入：

- 1 個月趨勢
- 3 個月趨勢
- 6 個月趨勢
- 1 年趨勢

## 7.12 初步資料表

### Stock

Id  
Market  
Ticker  
Name  
IsActive

唯一條件：

Market + Ticker

### StockGroup

Id  
GroupKey  
Name  
Description  
IsActive

### StockGroupMember

Id  
StockGroupId  
StockId  
EffectiveFrom  
EffectiveTo  
Source

用途：

- 表達股票屬於哪個族群。
- 支援成員關係的生效及失效日期。
- 記錄資料來源。

#### DailyStockTrading

TradingDate  
StockId  
OpenPrice  
HighPrice  
LowPrice  
ClosePrice  
ChangePercent  
TradingVolume  
TradingValue  
TransactionCount

#### 核心使用欄位：

- TradingDate
- StockId
- ClosePrice
- TradingValue

#### DailyGroupMetric

TradingDate  
StockGroupId  
TotalTradingValue  
MarketShare  
GroupReturn  
AdvancingRatio  
Top3Concentration  
MemberCount

#### 建議額外保存：

MembershipHash  
CalculatedAt

#### 用途：

- 保留當天族群成員版本。
- 確認族群成分何時發生變化。
- 避免歷史計算因目前族群名單改變而漂移。



## DailyGroupStockContribution

TradingDate  
StockGroupId  
StockId  
TradingValue  
ContributionRatio  
ReturnPercent

用途：

- 保存每天每個成分股對族群成交值的貢獻。
- 回頭查詢某個日期到底是哪幾檔股票帶動族群。

### 資料匯入紀錄

先前已提出需要保存資料匯入紀錄，但資料表名稱與欄位尚未正式確認。

預期至少記錄：

資料日期  
資料來源  
匯入時間  
匯入筆數  
是否成功  
錯誤訊息

## 7.13 歷史一致性

這是本功能的重要設計要求。

問題情境：

1. 今天將某檔股票加入「機器人」族群。
2. 若系統每次都使用今天的成分股名單重新計算過去資料。
3. 過去幾個月的機器人族群成交值與排名會被改寫。
4. 相同日期的歷史排行可能每次查詢都不同。

建議處理方式：

每日計算完成後，將結果固化到 SQLite：

- 每日族群統計
- 每日族群排名或可由每日指標排序重建
- 每日成分股貢獻

- 當日成員數
- 當日成員清單 Hash
- 計算時間

歷史結果不應在每次開啟網頁時，使用最新族群成員重新計算。

## 7.14 Google Sheets API 認證

先前建議的第一階段方式：

- 建立 Google Cloud Project。
- 啟用 Google Sheets API。
- 建立 Service Account。
- 將 Google Sheet 分享給 Service Account 的電子郵件。
- 權限先給 Viewer。
- 憑證只放後端。
- 不要將憑證 Commit 到 GitHub。
- 使用 `dotnet user-secrets` 或安全的本機設定保存憑證路徑。

此部分尚未實際執行。

## 7.15 Google Sheets 讀取策略

不應每切換一個畫面就重新呼叫 Google Sheets。

建議：

程式啟動或手動刷新

↓

一次讀取需要的 Sheet/Range

↓

轉換成 C# Model

↓

放入記憶體快取

↓

網頁查詢快取資料

第一版可以：

- 啟動時讀取一次。
- 提供「重新整理 Google Sheets」按鈕。
- 暫時不做高頻背景同步。

## 7.16 已討論但目前暫緩的資料模型

完整投資研究知識庫原本討論過：

- `Company`
- `Category`
- `CompanyCategory`
- `CategoryRelations`
- `NewsEvent`
- `NewsSource`
- `NewsCategory`
- `NewsCompany`
- `ResourceLink`
- `LegacyOverrides`

此模型是為了解決：

- 公司與族群多對多
- 公司與概念多對多
- 族群與概念關聯
- 一個事件多個新聞來源
- 研究連結查詢
- 舊 Sheet 資料覆寫

目前使用者已要求先暫緩概念股，因此不應在第一版優先實作上述完整模型。

---

## 8. 尚未解決問題

### 產品定義

1. 「成交值排行榜」的第一排序預設應是：
  - 絕對成交值
  - 成交值增幅
  - 市場成交比增幅
  - 綜合分數尚未最終決定。
2. 首頁預設期間尚未決定：
  - 5 日
  - 10 日
  - 20 日
3. 是否需要「1 日排行」尚未最終確認。

4. 「族群在動」是否需要綜合條件判斷，尚未正式定義。

可能條件包括：

- 排名快速上升
- 成交值放大
- 市場成交比提高
- 多數成分股同步上漲
- 集中度不能過高

但目前不建議一開始建立不透明的 AI 或綜合分數。

## 指標算法

5. 族群期間報酬率如何計算尚未確認：

- 等權
- 市值加權
- 成交值加權
- 族群指數

6. 股票在期間內有停牌、無成交或新上市時，如何處理尚未決定。

7. 上漲家數比應用：

- 期間起點到終點漲跌
- 每日上漲天數
- 期間平均日漲跌

尚未確認。

8. 前期比較是否永遠採用緊鄰的同長度區間，尚未正式確認。

9. 成交值的單位：

- 元
- 萬元
- 億元

只影響顯示，但尚未決定 UI 格式。

## 族群資料

10. Google Sheets 中族群資料目前的實際欄位與排列方式尚未整理。

11. 是否允許同一股票重複出現在同一族群尚未決定，需要建立去重規則。

12. 同一股票跨多族群是允許的，但 UI 需提醒市場成交比不能加總為 100%。

13. 族群名稱修改時，GroupKey 是否保持不變尚未正式確認。

14. 族群成分股的生效日期是否由 Google Sheets 維護，或由同步程式產生，尚未決定。

## 行情資料

15. TWSE 與 TPEX 的實際 API Endpoint 尚未選定。
16. 需確認官方資料能否一次取得：
  - 開盤價
  - 最高價
  - 最低價
  - 收盤價
  - 成交股數
  - 成交金額
  - 成交筆數
  - 漲跌幅
17. 歷史行情回補方式尚未確認。
18. 第一版回補天數尚未最終決定，先前建議 120~250 個交易日。
19. 缺漏日期、API 失敗及重複匯入的處理方式尚未設計。
20. 上市與上櫃是否合併成同一市場總成交值，或提供分市場比較，尚未正式決定。

## 系統與部署

21. 專案名稱尚未決定。
22. GitHub Repository 尚未建立。
23. SQLite 資料庫檔案路徑尚未決定。
24. 是否只在 Mac 本機使用，或未來需要手機／外部網路存取，尚未最終決定。
25. 每日資料更新的觸發方式尚未決定：
  - 手動按鈕
  - 程式啟動時
  - `BackgroundService`
  - macOS 排程

第一版較適合先做手動更新。

---

## 9. 限制、偏好與注意事項

### 回答語言與形式

- 使用繁體中文。
- 教學需明確、具體。
- 不要只給概念，要說明實際操作步驟。
- 不要假設使用者熟悉 macOS Terminal。
- 指令需要逐行說明。
- 程式碼需標明檔案名稱與路徑。
- 每次不要一次導入太多新技術。
- 優先完成能執行的最小版本。

## 技術偏好

- 優先使用 C#。
- 優先使用使用者已有經驗的物件導向與分層思維。
- VS Code 操作需盡可能對照 Visual Studio 的概念。
- 第一版避免前端 JavaScript 框架。
- 第一版避免 Docker 和資料庫伺服器。
- 不要一開始過度設計多專案架構。
- 不要一開始做微服務。
- 不要先追求漂亮 UI。

## 資料原則

- 不要讓 Google Sheets 和資料庫雙向同步同一筆資料。
- Google Sheets 暫時用於族群及成分股定義。
- SQLite 保存時間序列及每日計算結果。
- 歷史族群結果必須保持一致。
- 不應使用目前最新族群名單重算所有歷史。
- 同一股票可以屬於多個族群。
- 族群成交比可能重疊，不可假設總和為 100%。

## 開發原則

優先順序：

1. 計算正確
2. 資料可追溯
3. 歷史結果穩定
4. UI 可查詢
5. 最後才是視覺效果

---

## 10. 相關檔案與連結

## Google Sheets

<https://docs.google.com/spreadsheets/d/1rZpWz5GSHkvjoXlp5mX6SKoe2Yhn0MMfNfiiqTpDpZl/edit?usp=sharing>

用途：

- 目前投資資料來源。
- 暫定繼續維護族群及股票成分。
- 後續需確認分享權限與 API 存取方式。

## AI 智慧產業地圖參考網站

<https://aistockmap.com/?topic=advanced-packaging-test&activeTab=structure>

用途：

- 參考族群／產業結構頁面呈現。
- 使用者因此決定暫緩自行開發完整概念股知識圖譜。
- 不代表要直接複製其功能或資料。

## 預計官方行情來源

臺灣證券交易所：

<https://www.twse.com.tw/>

TWSE OpenAPI：

<https://openapi.twse.com.tw/>

證券櫃檯買賣中心：

<https://www.tpex.org.tw/>

TPEX OpenAPI：

<https://www.tpex.org.tw/openapi/>

以上 Endpoint 尚未實際選定與驗證。

---

## 11. 建議下一步

第一階段：確認 MVP 定義

下一個 ChatGPT 應先協助使用者確認第一版最小功能，不要立即串接 API。

建議第一版只做：

- 3 個測試族群
- 每個族群 5~10 檔股票
- 約 20~30 個交易日測試資料
- 5 日／10 日／20 日切換
- 平均成交值
- 成交值增減率
- 排名變化
- 市場成交比
- 上漲家數比
- 前三大集中度
- 成分股貢獻排行

## 第二階段：建立 macOS 開發環境

依小白教學方式逐步執行：

1. 檢查 Homebrew 是否安裝。
2. 檢查 Git。
3. 安裝／確認 .NET 10 SDK。
4. 安裝 VS Code。
5. 安裝 C# Dev Kit。
6. 在 Terminal 執行 `dotnet --info`。
7. 建立空白 Blazor Web App。
8. 用 VS Code 開啟專案。
9. 成功執行 localhost。
10. 建立 Git Repository。

每個步驟都應等使用者貼出執行結果後再繼續，以便排除環境問題。

## 第三階段：建立測試資料與公式

先不要串 TWSE、TPEX 或 Google Sheets。

先以程式內建或 JSON／CSV 測試資料驗證：

- 期間切割
- 本期與前期比較
- 排名計算



- 排名升降
- 市場成交比
- 集中度
- 成分股貢獻

確認公式正確後，再導入正式資料。

## 第四階段：建立 SQLite

完成：

- 安裝 EF Core SQLite 套件。
- 建立 `AppDbContext` 。
- 建立資料模型。
- 建立第一個 Migration 。
- 產生 SQLite 資料庫。
- 用 DB Browser 檢查資料。

## 第五階段：實作排行榜頁

第一版 UI：

日期

期間按鈕：5／10／20／60

排行模式

市場篩選

族群排行榜表格

點擊族群進入明細頁，顯示：

- 指標摘要
- 成分股貢獻表

暫時不做圖表。

## 第六階段：讀取 Google Sheets

完成：

- 整理 Google Sheets 族群欄位。
- 建立 `GroupKey` 。
- 建立 Google Cloud Project 。
- 啟用 Google Sheets API 。
- 建立 Service Account 。
- 設定 Viewer 權限。

- 建立 `GoogleSheetsClient` 。
- 讀取族群與成分股。
- 加入手動重新整理功能。

## 第七階段：串接官方行情

完成：

- 驗證 TWSE Endpoint 。
- 驗證 TPEX Endpoint 。
- 下載單日行情。
- 標準化上市及上櫃資料。
- 寫入 `DailyStockTrading` 。
- 建立重複匯入保護。
- 建立失敗紀錄。

## 第八階段：歷史回補與每日計算

- 回補 120~250 個交易日。
- 每日計算 `DailyGroupMetric` 。
- 每日保存 `DailyGroupStockContribution` 。
- 保存 `MemberCount` 。
- 保存 `MembershipHash` 。
- 加入手動更新按鈕。
- 後續再評估自動排程。

---

## 12. 給下一個 ChatGPT 的執行指示

1. 使用繁體中文回答。
2. 將使用者視為：
  - 熟悉 C# / WPF 的工程師。
  - 但不熟悉 macOS、VS Code、Blazor、EF Core 和網頁開發流程。
3. 不要重新把專案導向完整概念股或知識圖譜。
4. 當前第一優先是：

台股族群成交值排行榜與 5 / 10 / 20 / 60 日排行變化。
5. Google Sheets 目前保留作為族群及成分股定義來源。
6. SQLite 用於保存：
  - 每日行情
  - 每日族群指標

- 歷史排名所需資料
- 成分股貢獻
- 匯入紀錄

## 7. 第一版技術組合：

macOS  
VS Code  
C# Dev Kit  
.NET 10  
Blazor Web App Interactive Server  
ASP.NET Core  
Entity Framework Core  
SQLite  
Google Sheets API  
Git/GitHub

## 8. 不要一開始加入：

React  
TypeScript  
Docker  
PostgreSQL  
Redis  
微服務  
盤中即時行情  
AI 綜合評分  
新聞系統

## 9. 開發時採小步驟：

建立環境  
→ 跑起空白網頁  
→ 測試資料  
→ 計算公式  
→ SQLite  
→ 排行表  
→ Google Sheets  
→ TWSE/TPEX  
→ 歷史回補

## 10. 每次提供指令時都需說明：

- 在哪裡輸入。
- 指令用途。
- 預期輸出。
- 發生錯誤時要貼出哪些資訊。

## 11. 尚未確定的算法不能自行假設已定案，尤其是：

- 族群報酬率算法。

- 預設排行模式。
  - 預設觀察期間。
  - 上漲家數比的期間定義。
  - TWSE／TPEX 的實際 Endpoint。
  - 每日更新觸發方式。
12. 在開始撰寫正式程式前，應先協助使用者確認：
    - 第一版排行表欄位。
    - 5／10／20 日的期間比較方式。
    - Google Sheets 族群資料目前實際格式。
    - 是否先使用手工測試資料驗證公式。
  13. 不要宣稱目前已有任何程式、資料庫或 API 串接成果；目前只有需求與架構討論。
  14. 最適合接續本對話的下一個任務是：

從 macOS + VS Code 開始，逐步建立並成功執行第一個 Blazor Web App 空白專案。